

## Journal Name

Crossmark

TAREA 7

RECEIVED  
dd Month yyyyREVISED  
dd Month yyyy

## Aplicación de Kernel con Convolución

Daniel González<sup>1</sup> <sup>1</sup>Facultad de Ingeniería, Universidad Autónoma de Querétaro, Querétaro, México  
16/10/2025**E-mail:** dgonzalez16@alumnos.uaq.mx**Keywords:** Python, kernel, convolución, preprocesamiento, imagenes, filtros**Abstract**

En este trabajo se presenta la aplicación de filtros basados en kernels a una imagen utilizando el método de convolución, con la librería OpenCV de Python como de forma manual. Se busca demostrar el funcionamiento de la operación de convolución y analizar su efecto sobre la imagen original al modificar los valores del kernel. Mediante la comparación entre ambos enfoques, se verificó la similitud en los resultados además de demostrar de manera clara como se aplican los filtros más allá de solo usar una función la cual nos permite resaltar bordes, suavizar o realzar detalles en imágenes digitales.

**1 Introducción**

El reconocimiento de patrones y el procesamiento de imágenes frecuentemente requieren operaciones de filtrado para resaltar características relevantes. La convolución con kernels pequeños (por ejemplo  $3 \times 3$ ) es una operación fundamental para detectar bordes, suavizar ruido o realzar detalles. En este reporte se describen los kernels utilizados, la implementación en Python con OpenCV y la implementación manual, y se muestran resultados sobre imágenes de ejemplo.

**2 Materiales y métodos***2.1 Kernels empleados*

A continuación se listan los kernels usados en la libreta de Python. Se definen como matrices  $3 \times 3$ :

**Listing 1.** Kernels usados (ejemplo)

$$K1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$K2 = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

$$K3 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$K4 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

$$K5 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$K6 = (1/16) * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

K3 y K4 son kernels Laplacianos para detección de bordes; K5 es un realce (sharpen) y K6 un suavizado tipo gaussiano.

### 3 Desarrollo

#### 3.1 Implementación con OpenCV

OpenCV ofrece la función `cv2.filter2D` (`src`, `ddepth`, `kernel`) que realiza la convolución de forma sencilla. Es importante mencionar que la librería de OpenCV separa los canales BGR (Blue, Green, Red) diferente al típico RGB (Red, Green, Blue), se aplica `filter2D` por canal y luego se normaliza el resultado para visualizarlo.

Código:

```
1 B, G, R = cv2.split(img32)
2
3 resp_k1_b = cv2.filter2D(B, cv2.CV_32F, K1)
4 resp_k1_g = cv2.filter2D(G, cv2.CV_32F, K1)
5 resp_k1_r = cv2.filter2D(R, cv2.CV_32F, K1)
6 resp_k1_b = cv2.normalize(np.abs(resp_k1_b), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
7 resp_k1_g = cv2.normalize(np.abs(resp_k1_g), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
8 resp_k1_r = cv2.normalize(np.abs(resp_k1_r), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
9 out_k1_bgr = cv2.merge([resp_k1_b, resp_k1_g, resp_k1_r])
```

Figure 1. Uso de OpenCV para aplicar filtros

#### 3.2 Implementación manual

La convolución manual consiste en:

1. Calcular padding apropiado (reflect o zero).
2. Para cada píxel (y,x) y cada canal: extraer la vecindad del tamaño del kernel.
3. Realizar la suma elemento a elemento entre la vecindad y el kernel.
4. Guardar el valor en la salida; aplicar post-procesado (valor absoluto y normalización para visualizar).

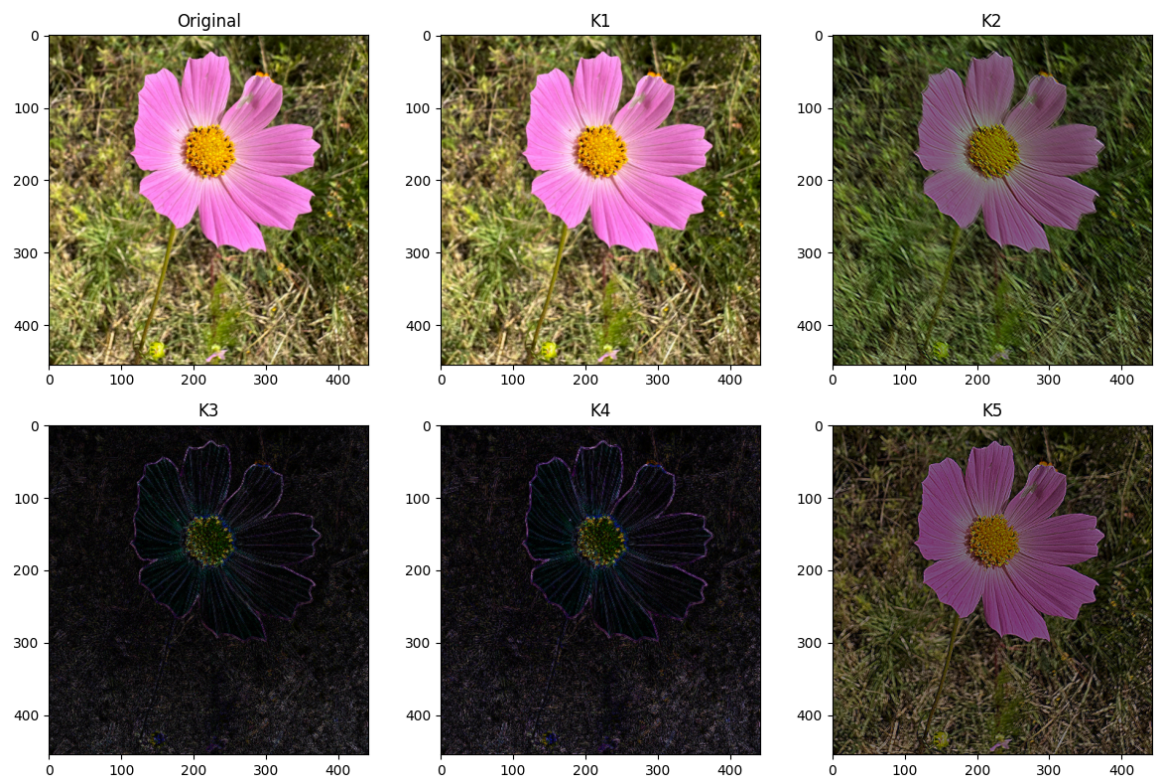
Código:

```
1 kh, kw = K3.shape
2 pad_h, pad_w = kh // 2, kw // 2
3
4 # Padding
5 padded = np.pad(img_np, ((pad_h, pad_h), (pad_w, pad_w), (0,0)), mode='reflect')
6
7 # Salida
8 out = np.zeros_like(img_np)
9
10 for y in range(alt):
11     for x in range(anc):
12         for ch in range(c): # B, G, R
13             region = padded[y:y+kh, x:x+kw, ch] # región de la imagen del tamaño del kernel
14             out[y, x, ch] = np.sum(region * K3) # suma ponderada
15
16 out = np.abs(out)
17 out = (out - out.min()) / (out.max() - out.min()) * 255
18 out = out.astype(np.uint8)
```

Figure 2. Implementación manual de convolución

## 4 Resultados

### 4.1 *OpenCV*



**Figure 3.** Resultados con OpenCV

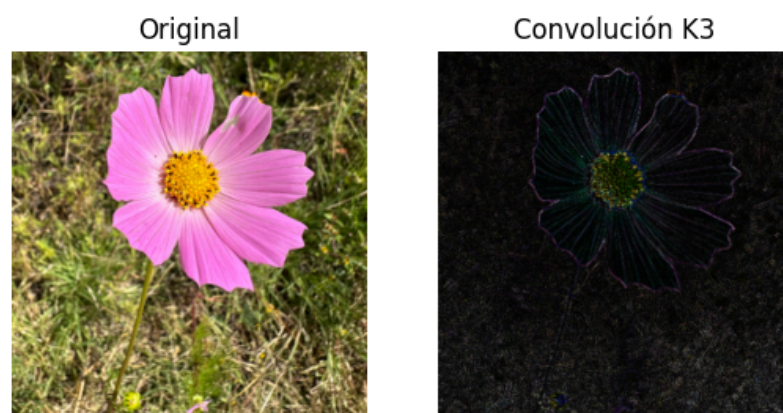
### 4.2 *Implementación manual*



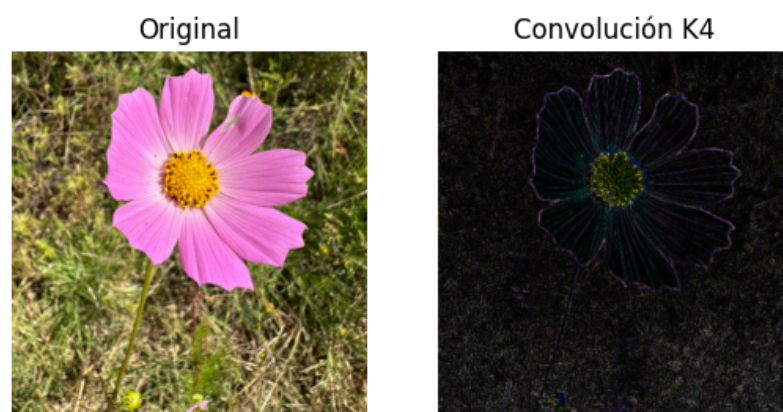
**Figure 4.** Resultados con implementación manual (K1)



**Figure 5.** Resultados con implementación manual (K2)



**Figure 6.** Resultados con implementación manual (K3)



**Figure 7.** Resultados con implementación manual (K4)



**Figure 8.** Resultados con implementación manual (K5)



**Figure 9.** Resultados con implementación manual (K6)

## 5 Conclusión

Se implementaron filtros basados en kernels mediante convolución tanto con OpenCV como de forma manual. Los resultados obtenidos fueron consistentes entre ambos métodos, validando la correcta aplicación de la operación de convolución. La manipulación de los valores del kernel permitió observar efectos variados en la imagen, como detección de bordes, realce y suavizado. Con esta prueba podemos observar además del comportamiento de la librería OpenCV (Implementación rápida y sencilla), la importancia de los parámetros en la convolución y cómo afectan el resultado final, así como la vista paso a paso de la operación al implementarla manualmente.